

navigator.digitclassifier: an end-to-end walkthrough

A file-by-file trace of the experimental `navigator.digitclassifier` Web API on the `digitclassifier` branch. Everything below was read out of the source at `a382504ce8a40`; where a claim came from a comment or an existing doc rather than from code, it is called out as such. The last section lists the places where the in-tree docs and comments are wrong.

Source links. Paths below link into github.com/obeletski/chromium on the `digitclassifier` branch. Ambiguous basenames, build output under `out/`, and anything under `third_party/dawn/` are left unlinked — dawn is a git submodule, so its files are not in this repository.

1. What the feature is

A non-standard, flag-gated Web API that classifies a hand-drawn digit. The whole JS surface is two calls:

```
const model = await navigator.digitclassifier.getModel();
const digit = await model.classify(inputBuffer); // "0" .. "9"
```

`inputBuffer` is a `Float32Array` of exactly **2352** values — 28 x 28 x 3 in NHWC order, each in `[0,1]`. The three channels exist because the bundled model was trained on RGB input; callers replicate one grayscale value across all three (`tools/smoke_test.html:23`, `nhwc()`).

`classify()` resolves with a **DOMString**, and **nothing else**. There is no confidence, no probability vector, no logits. That is deliberate: softmax is never computed anywhere in the shipped path, because `argmax(softmax(z))` equals `argmax(z)` and only the winner is reported (see §7).

Three IDL files define the surface.

`third_party/blink/renderer/modules/digitclassifier/navigator_digit_classifier.idl`:

```
[
  RuntimeEnabled=DigitClassifier
] interface mixin NavigatorDigitClassifier {
  [SameObject, SecureContext] readonly attribute DigitClassifier digitclassifier;
};

Navigator includes NavigatorDigitClassifier;
```

`digit_classifier.idl`:

```
[
  Exposed=Window,
  RuntimeEnabled=DigitClassifier,
  SecureContext
] interface DigitClassifier {
  [CallWith=ScriptState, RaisesException]
  Promise<DigitClassifierModel> getModel();
};
```

`digit_classifier_model.idl`:

```
[
  Exposed=Window,
  RuntimeEnabled=DigitClassifier,
  SecureContext
] interface DigitClassifierModel {
  [CallWith=ScriptState, RaisesException]
  Promise<DOMString> classify(Float32Array input);
};
```

Three things to read off these:

- **Exposed=Window only.** No worker exposure. This is load-bearing later: the execution context is always a `LocalDOMWindow`, which is why `digit_classifier.cc` can `To<LocalDOMWindow>(context)->document()->Token()` unconditionally.
- **[SecureContext] on all three.** The API does not exist on an insecure origin, which is why the on-device smoke test has to be served over HTTP from the device's own `127.0.0.1` rather than from `file://`.
- **RuntimeEnabled=DigitClassifier**, declared at `third_party/blink/renderer/platform/runtime_enabled_features.json5:2280`:

```
{
  name: "DigitClassifier",
  status: "test",
},
```

The json5 header (line 23) defines `status: "test"` as *"Enabled in ContentShell for testing, otherwise off."* In Chrome it therefore needs `-enable-blink-features=DigitClassifier` on the command line. This is the honest setting for an API with no spec and no chromestatus entry.

2. The moving parts

File	Job
<code>navigator_digit_classifier.{h,cc}</code>	Supplement<NavigatorBase> that lazily creates the one <code>DigitClassifier</code> per navigator
<code>digit_classifier.{h,cc}</code>	<code>getModel()</code> : context provider -> Dawn client -> adapter -> device -> weights -> model
<code>digit_classifier_weights.{h,cc}</code>	Loads and validates <code>IDR_DIGIT_CLASSIFIER_WEIGHTS</code> , hands out four <code>base::span<const float></code>
<code>digit_classifier_model.{h,cc}</code>	The WGSL source, pipeline/bind-group construction, <code>classify()</code> dispatch and readback
<code>resources/digit_classifier_weights.bin</code>	1,209,928 bytes of header + float32 weights
<code>tools/</code>	Offline only: tflite parsing, weight extraction, the CPU oracle, the smoke-test page and server

Nothing here is compiled into a separate service and there is no Mojo interface. The renderer drives Dawn directly over the existing WebGPU wire.

3. The call path, in execution order

3.1 JS -> generated V8 bindings

`navigator.digitclassifier` is an attribute on the `Navigator` interface, supplied by the mixin. The bindings generator emits `v8_digit_classifier.{cc,h}` and `v8_digit_classifier_model.{cc,h}` into `$root_gen_dir/third_party/blink/renderer/bindings/modules/v8/`. The mixin itself produces no `v8_navigator_digit_classifier.*` pair — the attribute is folded into the generated `Navigator` bindings, which is why `generated_in_modules.gni` lists **four** generated files for **three** source IDLs.

The generated attribute getter calls the static `NavigatorDigitClassifier::digitclassifier(NavigatorBase&)`; the generated method callbacks call `DigitClassifier::getModel(ScriptState*, ExceptionState&)` and `DigitClassifierModel::classify(ScriptState*, NotShared<DOMFloat32Array>, ExceptionState&)`. `[CallWith=ScriptState]` adds the leading `ScriptState*`; `[RaisesException]` adds the trailing `ExceptionState&`.

3.2 `navigator_digit_classifier.cc`

Standard supplement boilerplate, 33 lines. The detail worth knowing is that the `DigitClassifier` is created **eagerly in the supplement's constructor**, not on first `getModel()`:

```
NavigatorDigitClassifier::NavigatorDigitClassifier(NavigatorBase& navigator)
: Supplement<NavigatorBase>(navigator),
  digit_classifier_(MakeGarbageCollected<DigitClassifier>(
    navigator.GetExecutionContext())) {}
```

The supplement itself is lazy (`Supplement<NavigatorBase>::From<...> then ProvideTo`), so nothing happens until script touches the attribute. Constructing a `DigitClassifier` allocates no GPU resources — it only stores the execution context — so `[SameObject]` is cheap to honour.

Note the base class is `Supplement<NavigatorBase>`, not `Supplement<Navigator>` as `DigitClassifier-Handoff.md` states.

3.3 digit_classifier.cc — getModel()

```
ScriptPromise<DigitClassifierModel> DigitClassifier::getModel(
    ScriptState* script_state, ExceptionState& exception_state) {
    if (!script_state->ContextIsValid()) { /* throw InvalidStateError */ }

    auto* resolver = MakeGarbageCollected<ScriptPromiseResolver<DigitClassifierModel>>(
        script_state, exception_state.GetContext());
    auto promise = resolver->Promise();

    EnsureDawnControlClient(
        script_state,
        BindOnce(&DigitClassifier::RequestDevice, WrapPersistent(this),
            WrapPersistent(script_state), WrapPersistent(resolver)));

    return promise;
}
```

The promise is created and returned immediately; everything else runs on callbacks. `BindOnce` here is `blink::BindOnce` (from `platform/wtf/functional.h`), not `WTF::BindOnce` and not `base::BindOnce`.

3.4 EnsureDawnControlClient() — the WebGPU context

This is the interesting function and the one both fixed bugs lived in.

```
if (dawn_control_client_ && !dawn_control_client_->IsContextLost()) {
    std::move(callback).Run();
    return;
}
dawn_control_client_ready_callbacks_.push_back(std::move(callback));
if (dawn_control_client_ready_callbacks_.size() > 1) {
    return; // a creation request is already in flight
}
```

The queue-and-coalesce pattern is copied from `GPU::RequestAdapterImpl()` in `third_party/blink/renderer/modules/webgpu/gpu.cc`, which does the same with `dawn_control_client_initialized_callbacks_`. Concurrent `getModel()` calls therefore share one context-provider creation.

Then:

```
CreateWebGPUGraphicsContext3DProviderAsync(
    execution_context->Url(), Platform::WebGPUReplyThread::kMainThread,
    execution_context->GetTaskRunner(TaskType::kWebGPU),
    CrossThreadBindOnce([(CrossThreadHandle<DigitClassifier> classifier_handle,
        CrossThreadHandle<ExecutionContext> execution_context_handle,
        std::unique_ptr<WebGraphicsContext3DProvider> context_provider) {
        ...
        if (context_provider && context_provider->BindToCurrentSequence()) {
            context_provider->WebGPUInterface()->SetWebGPUExecutionContextToken(
                ToLocalDOMWindow<(context)->document()->Token());
            classifier->dawn_control_client_ = DawnControlClientHolder::Create(
                std::move(context_provider), context->GetTaskRunner(TaskType::kWebGPU));
        }
        classifier->OnDawnControlClientReady();
    }], MakeCrossThreadHandle(this), MakeCrossThreadHandle(execution_context)));
```

Things that are easy to get wrong here:

- The lambda is a `CrossThreadBindOnce` taking `CrossThreadHandle<>` s, unwrapped with `MakeUnwrappingCrossThreadHandle(...)` then `GetOnCreationThread()`. If either unwrap fails the lambda returns **without draining** `dawn_control_client_ready_callbacks_` — a pending promise.
- `WebGPUReplyThread::kMainThread` is hardcoded; `gpu.cc` picks `kIOThread` for multithreaded workers, which `Exposed=Window` never needs.
- DigitClassifier skips a check that WebGPU performs.** `gpu.cc` routes the provider through `CheckContextProvider()`, which first asks `mojom::blink::GpuDataManager::Are3DAPISBlockedForUrl()` and drops the provider if 3D APIs are blocked for the origin. `digit_classifier.cc` calls `BindToCurrentSequence()` directly and never consults `GpuDataManager`. That divergence appears in none of the existing docs.

`OnDawnControlClientReady()` then moves the queue out and runs every closure. If the provider failed, `dawn_control_client_` is still null and the queued `RequestDevice` will reject rather than hang — that is what makes "pending forever" a diagnosable signature (see §8).

3.5 Adapter and device

`RequestDevice()` rejects with `OperationError` if there is no client, then:

```
wgpu::RequestAdapterOptions options = {}; // any adapter will do
auto* callback = MakeWGPUOnceCallback(resolver->WrapCallbackInScriptScope(
    BindOnce(&DigitClassifier::OnRequestAdapter, WrapPersistent(this),
        WrapPersistent(script_state))));
dawn_control_client_>GetWGPUInstance().RequestAdapter(
    &options, wgpu::CallbackMode::AllowProcessEvents,
    callback->UnboundCallback(), callback->AsUserData());
dawn_control_client_>EnsureFlush(*execution_context->GetAgent()->event_loop());
```

No power preference, no feature level, no fallback adapter — the default descriptor. The `EnsureFlush()` call is mandatory: `AllowProcessEvents` means the reply is delivered when the event loop processes Dawn events, and without the flush the wire commands can sit in the client's buffer.

`WrapCallbackInScriptScope()` **injects the resolver as the first unbound parameter**, so every bound argument must come before it. That is why `OnRequestAdapter(script_state, resolver, status, adapter, error_message)` binds only `this` and `script_state`.

`OnRequestAdapter()` rejects on non-success or a null adapter, otherwise calls `adapter.RequestDevice()` with an empty `wgpu::DeviceDescriptor`, the same callback machinery and another `EnsureFlush()`. Error strings arrive as `wgpu::StringView`; the file-local `StringViewToString()` helper exists because the conversion operator resolves the `WGPU_STRLen` sentinel, so an absent message arrives as an *empty* view, and both handlers substitute a generic string for it.

3.6 `digit_classifier_weights.cc` — loading the blob

`OnRequestDevice()` is where the weights are read, **after** the device exists:

```
std::optional<DigitClassifierWeights> weights = DigitClassifierWeights::Load();
```

`Load()` does five things, in order:

1. `UncompressResourceAsBinary(IDR_DIGIT_CLASSIFIER_WEIGHTS) -> Vector<char>`.
2. Exact size check: `kHeaderSize (32) + kTotalFloatCount * sizeof(float)`, i.e. 1,209,928 bytes. Not a minimum — an exact equality.
3. Magic check against `{'D','C','L','S'}` via `std::ranges::equal`.
4. `version == 1`.
5. The three declared shapes must equal the compiled-in `kInputSize (2352)`, `kHiddenSize (128)`, `kOutputSize (10)`:

```
// The shader is compiled against these sizes, so a blob describing a
// different network must be rejected rather than reinterpreted.
if (base::U32FromLittleEndian(bytes.subspan<8u, 4u>()) != kInputSize || ...
```

Then the payload is copied into a `Vector<float>`:

```
Vector<float> storage(static_cast<wtf_size_t>(kTotalFloatCount));
base::as_writable_bytes(base::allow_nonunique_obj, base::span(storage))
    .copy_from(bytes.subspan(kHeaderSize));
```

`base::allow_nonunique_obj` is required because `float` has non-unique object representations (NaN payloads, signed zero), which `CanSafelyConvertToByteSpan` rejects by default. Every failure path returns `std::nullopt` after a `DLOG(ERROR)` — so on a release build a malformed blob produces the generic "missing or malformed" rejection with no log line.

The four accessors are plain subspans over one contiguous `Vector<float>`: `layer1_weights()` (301,056 floats), `layer1_bias()` (128), `layer2_weights()` (1,280), `layer2_bias()` (10).

Two consequences: `Load()` **runs on every `getModel()` call** (no caching), and the load is *not* allocation-free despite what the `.grd` comment claims — §11.

3.7 `digit_classifier_model.cc` — building the pipeline

`DigitClassifierModel::Create()` allocates the object and calls `Initialize()`, returning `nullptr` if that fails; `OnRequestDevice()` turns `nullptr` into an `OperationError` rejection, otherwise `resolver->Resolve(model)`.

`Initialize()` creates six buffers. Four are weight buffers built by the file-local helper:

```
descriptor.usage = wgpu::BufferUsage::Storage | wgpu::BufferUsage::CopyDst;
wgpu::Buffer buffer = device.CreateBuffer(&descriptor);
if (buffer) { queue.WriteBuffer(buffer, 0, data.data(), data.size_bytes()); }
```

plus `input_buffer_` (2352 floats, `Storage | CopyDst`) and `result_buffer_` (4 bytes, `Storage | CopySrc`). Then the shader module, the compute pipeline, and a bind group whose layout comes from `pipeline_.GetBindGroupLayout(0)` — the implicit layout derived from the WGSL, not a hand-written `BindGroupLayoutDescriptor`.

The shader is a single string constant, `kShaderSource`, and it is the whole network:

```
@group(0) @binding(0) var<storage, read> w1 : array<f32>;
...
@group(0) @binding(5) var<storage, read_write> out_data : array<u32>;

var<workgroup> hidden : array<f32, 128>;
var<workgroup> logits : array<f32, 10>;

@compute @workgroup_size(128)
fn main(@builtin(local_invocation_id) lid : vec3<u32>) {
    let j = lid.x;
    var acc = b1[j];
    let row = j * kInput;
    for (var k = 0u; k < kInput; k = k + 1u) {
        acc = acc + w1[row + k] * in_data[k];
    }
    hidden[j] = max(acc, 0.0);

    workgroupBarrier();

    if (j < kOutput) { /* layer 2 into logits[j] */ }

    workgroupBarrier();

    if (j == 0u) { /* argmax over logits, out_data[0] = best */ }
}
```

Design points that are only obvious once you read it:

- **One workgroup, one dispatch.** `@workgroup_size(128)` equals `kHidden`, so invocation `j` owns hidden unit `j` and layer 1 needs no bounds check. At ~~302K MACs~~ (0.6 MFLOP) dispatch and readback dominate, so minimising round trips beats parallelism. Workgroup storage is `128*4 + 10*4 = 552` bytes against a 16 KiB floor.
- **Both `workgroupBarrier()` calls sit in uniform control flow** — outside the `if (j < kOutput)` and `if (j == 0u)` blocks. A barrier inside non-uniform control flow is a WGSL validation error.
- **`max(acc, 0.0)` is scalar on purpose.** Dawn enables `Toggle::ScalarizeMaxMinClamp` on Qualcomm (crbug 407109052), but `ShouldAttemptScalarize()` in `third_party/dawn/src/tint/lang/core/ir/transform/builtin_scalarize.cc` bails when the builtin's result type is not a `core::type::Vector` — so on a scalar ReLU the transform is a no-op today. Vectorising the dot product with `vec4` would make it live.
- **No softmax.** Confirmed monotonic-equivalence is asserted in `tools/reference_infer.py:137`.

3.8 `classify()`

Validation happens first, and all of it throws synchronously (the IDL declares `[RaisesException]`) rather than rejecting:

Condition	Thrown
<code>!script_state->ContextIsValid()</code>	<code>InvalidStateError</code>
<code>input->IsDetached()</code>	<code>TypeError</code>
<code>input->length() != 2352</code>	<code>TypeError</code> with both counts
<code>dawn_control_client->IsContextLost()</code>	<code>OperationError</code>
readback buffer allocation failed	<code>OperationError</code>

`DOMTypedArray::length()` returns `size_t`, hence `%zu` in the `String::Format`.

Then the per-call GPU work:

```
wgpu::Buffer readback = device_.CreateBuffer(&readback_descriptor); // MapRead|CopyDst, 4 B
queue.WriteBuffer(input_buffer_, 0, values.data(), values.size_bytes());
wgpu::CommandEncoder encoder = device_.CreateCommandEncoder();
{
    wgpu::ComputePassEncoder pass = encoder.BeginComputePass();
    pass.SetPipeline(pipeline_);
    pass.SetBindGroup(0, bind_group_);
    pass.DispatchWorkgroups(1);
    pass.End();
}
encoder.CopyBufferToBuffer(result_buffer_, 0, readback, 0, kResultSize);
queue.Submit(1, &commands);
```

The **readback buffer is freshly allocated per call** while the input, result and weight buffers are shared. The comment gives the reason: overlapping `classify()` calls are ordered by the queue for everything except a *mapped range*, which is not queue-ordered.

Finally `readback.MapAsync(..., AllowProcessEvents, ...)` with the same `MakeWGPUOnceCallback/WrapCallbackInScriptScope` sandwich, followed by `EnsureFlush()`. Note the bound argument order again: `BindOnce(&DigitClassifierModel::OnResultMapped, WrapPersistent(this), readback)` — `readback` is bound, `resolver` is injected after it.

3.9 OnResultMapped() — the result

```
const void* mapped = readback.GetConstMappedRange(0, kResultSize);
...
const auto bytes = UNSAFE_BUFFERS(
    base::span(static_cast<const uint8_t*>(mapped), kResultSize));
const uint32_t digit = base::U32FromLittleEndian(bytes.first<4u>());
readback.Unmap();

if (digit >= DigitClassifierWeights::kOutputSize) { /* reject */ }
resolver->Resolve(String::Number(digit));
```

Three rejection paths (map failure, null mapped range, out-of-range index) and one resolve. The `UNSAFE_BUFFERS` block is the only pointer-arithmetic escape hatch in the module, and it carries the `// SAFETY: justification that -Wunsafe-buffer-usage requires`. The result crosses the wire as a `u32` index, not as text — `String::Number()` is what turns `7` into `"7"`.

4. Why it is all async

Both calls are async because the work genuinely crosses processes, twice.

`getModel()` cannot be synchronous because it chains four asynchronous steps:

1. `CreateWebGPUGraphicsContext3DProviderAsync()` — the renderer asks the browser for a GPU channel.
2. `RequestAdapter` — a wire round trip into the GPU process. The service then **defers the reply until the browser has given it an isolation key** (`gpu/command_buffer/service/webgpu_decoder_impl.cc:1360`), so this is really two round trips.
3. `RequestDevice` — another wire round trip.
4. Resource load + pipeline creation, which is the only synchronous part.

`classify()` cannot be synchronous because the answer is behind a `MapAsync` on a buffer the GPU has not written yet. The dispatch is submitted, the queue drains on the GPU, the copy lands in a mappable buffer, and only then does the wire deliver the mapped memory back to the renderer.

The original design ask had both calls synchronous. That was rejected as impossible, and the rejection is recorded in `DigitClassifier-Handoff.md`. There is **no CPU fallback path** anywhere in the module, so a resolved `classify()` is proof that the WGSL was compiled by Tint, lowered to SPIR-V, and executed on the GPU.

5. The four registration points

A new Blink module is invisible until all four of these know about it. Missing any one produces a different, unhelpful failure.

File:line	Line	Failure
third_party/blink/renderer/modules/BUILD.gn:92	<code>"/third_party/blink/renderer/modules/digitclassifier",</code>	Module module
third_party/blink/renderer/bindings/idl_in_modules.gni:247-249	the three <code>.idl</code> paths	The bir IDL; no
third_party/blink/renderer/bindings/generated_in_modules.gni:2268-2271	the four <code>v8_digit_classifier*.{cc,h}</code> paths	Files ar fails lat bindir only on
third_party/blink/renderer/platform/runtime_enabled_features.json5:2280	<code>name: "DigitClassifier", status: "test"</code>	Runtin to reso

The second and third are the trap: [generated_in_modules.gni](#) is **not derived from** [idl_in_modules.gni](#), both must be edited by hand, and grepping the generated list for a source path finds nothing — which reads like the edit is unnecessary. A fifth edit, [blink_resources.grd:63](#), ships the data.

The module's own `BUILD.gn` needs two deps:

```
deps = [
  "//gpu/command_buffer/client/webgpu_interface",
  "//third_party/blink/public/resources",
]
```

The first was added by the bug-2 fix. Without it the code *compiles* but `gn check` fails, because [webgpu_interface.h](#) is otherwise reachable only through a private edge from `//third_party/blink/renderer/platform`.

6. How the weights get there

Offline

`digit_classifier.tflite` (1,212,797 bytes) sits untracked at the repo root and is deliberately never committed — it is a build *input*, and the extracted blob is what ships.

[tools/tflite_dump.py](#) is a stdlib-only flatbuffer reader (the checkout has neither `flatbuffers` nor `numpy` nor `schema.fbs`). It printed the architecture rather than letting anyone assume it: `RESHAPE -> [1,2352], FULLY_CONNECTED W[128,2352] b[128] act=RELU, FULLY_CONNECTED W[10,128] b[10] act=NONE, SOFTMAX`.

[tools/extract_weights.py](#) imports `tflite_dump` and pulls four named tensors in a fixed order, checking type, shape, buffer byte count, and NaN/Inf, then writes:

```
0  char[4]  magic "DCLS"
4  u32      version = 1
8  u32      input_size  = 2352
12 u32      hidden_size = 128
16 u32      output_size = 10
20 u32[3]   reserved (zero)
32 f32[128*2352] W1 (row-major)
   f32[128]   b1
   f32[10*128] W2 (row-major)
   f32[10]    b2
```

$32 + 302,474 * 4 = \mathbf{1,209,928 \text{ bytes}}$, which is the size on disk. TFLite stores FC weights row-major `[units, input_size]`, which is exactly the layout the shader indexes (`w1[j * kInput + k]`), so no transpose happens anywhere.

At runtime

[blink_resources.grd:63](#):

```
<include name="IDR_DIGIT_CLASSIFIER_WEIGHTS"
  file="../../renderer/modules/digitclassifier/resources/digit_classifier_weights.bin"
  type="BINDATA"/>
```

`IDR_DIGIT_CLASSIFIER_WEIGHTS` resolves to **45307** (confirmed in the generated `out/Default/gen/third_party/blink/public/resources/grit/blink_resources.h:35`), and `blink_resources.pak` is already packed into Chrome.

Why not an APK asset. The first plan used `android_assets` plus `base::android::OpenApkAsset()`. [base/android/apk_assets.h:18](#) says "Can be used from renderer process." **That comment is wrong for a sandboxed renderer:** [apk_assets.cc:29](#) calls `Java_ApkAssets_open()` — JNI into the Java `AssetManager` — and an Android renderer is an `isolatedProcess` with no such access. Child processes get file data as pre-opened FDs in a `FileDescriptorStore` at launch ([base/android/child_process_service.cc](#)), which is how ICU and the V8 snapshot arrive. APK assets would have forced browser<->renderer IPC back into a design built to avoid it.

The resource is deliberately **uncompressed**. Float32 weights are essentially incompressible, so compression would buy nothing and cost a decompression pass on every load.

7. The two bugs

Both lived in `EnsureDawnControlClient()`, both were found only by running on real hardware, and both are fixed and verified.

Bug 1 — renderer crash (a29ea95bd7e6c)

Symptom. The renderer died before any page script ran:

```
F/chromium: [FATAL:services/viz/public/cpp/gpu/context_provider_command_buffer.cc:565]
DCECK failed: bind_tried_.
```

Mechanism. Line 565 is the first line of `ContextProviderCommandBuffer::WebGPUInterface()`:

```
gpu::webgpu::WebGPUInterface* ContextProviderCommandBuffer::WebGPUInterface() {
    DCECK(bind_tried_);
    DCECK_EQ(bind_result_, gpu::ContextResult::kSuccess);
```

`DawnControlClientHolder`'s constructor reaches for it **from its initializer list**:

```
api_channel_(context_provider_>ContextProvider()
              .WebGPUInterface()
              .>GetAPIChannel()),
```

The original code handed the freshly created provider straight to `DawnControlClientHolder::Create()`, so nothing had ever called `BindToCurrentSequence()`.

Fix. One condition: `if (context_provider && context_provider->BindToCurrentSequence())`, treating failure as "no context" — which matches `CheckContextProvider()` in [modules/webgpu/gpu.cc:246](#).

Bug 2 — `getModel()` hangs forever (f1b678daad675)

Symptom. `getModel()` never settled: no resolve, no reject. The smoke test's 20-second `Promise.race` timer was the only thing that ended the wait, printing `getModel TIMEOUT`.

Mechanism. In [gpu/command_buffer/service/webgpu_decoder_impl.cc:1360](#):

```
if (isolation_key_) {
    callback_info.callback(WGPURequestAdapterStatus_Success, ...);
} else {
    // We can't RequestDevice until we have an isolation key. Defer the
    // RequestAdapter callback until we do.
    DCECK_NE(isolation_key_provider_, nullptr);
    deferred_request_adapter_callbacks_.emplace_back(...);
}
```

`isolation_key_` is populated only by `WebGPUDecoderImpl::OnGetIsolationKey()` (:2495), which is only ever scheduled from `HandleSetWebGPUExecutionContextToken()` (:2509, callback bound at :2553), which only fires when the *client* sends `SetWebGPUExecutionContextToken`. [modules/webgpu/gpu.cc](#) sends it; this module did not. So the service parked the `RequestAdapter` reply on `deferred_request_adapter_callbacks_` and nothing ever drained it.

Fix. Send the document token before constructing the Dawn client:

```
context_provider->WebGPUInterface()->SetWebGPUExecutionContextToken(
    To<LocalDOMWindow>(context)->document()->Token());
```

`DigitClassifier` is `Exposed=Window`, so the document token is the only case; `GetExecutionContextToken()` in [gpu.cc](#) covers the worker variants if that ever changes. The fix also added the `//gpu/command_buffer/client/webgpu_interface` dep for `gn` check.

The diagnostic worth remembering. Every error path in `digit_classifier.cc` rejects. So a promise that stays pending is never a swallowed failure — it is always a callback that never arrived. That distinction is what localised bug 2 to a deferred reply in the service rather than to anything in Blink.

8. How it is verified

The CPU oracle

`tools/reference_infer.py` is the ground truth. It is a **pure-stdlib Python forward pass** (`import math, struct, sys` — no NumPy), which mirrors the shader in the same order:

```
hidden[j] = relu(dot(W1[j], x) + b1[j])
logits[i] = dot(W2[i], hidden) + b2[i]
answer    = argmax(logits)      # softmax omitted, it is monotonic
```

It draws three synthetic digits with plain loops (`draw_one()`, `draw_seven()`, `draw_zero()`), replicates grayscale across channels with `nhwc_from_gray()`, and asserts the monotonicity property the shader depends on:

```
assert best == max(range(n_out), key=lambda i: probs[i])
```

Its expected outputs:

input	expected	confidence
synthetic 1	"1"	1.0000
synthetic 7	"7"	0.9593
synthetic 0	"0"	0.9984
blank	"5"	0.1428 (bias-only path; near-uniform is correct)

The blank case matters: it is the only one that exercises the bias-only path through both layers, and "5" at p=0.14 is the *correct* answer, not a failure.

The on-device smoke test

`tools/smoke_test.html` reproduces the same four inputs in JS — `drawOne()`, `drawSeven()`, `drawZero()` are line-for-line ports of the Python — and compares against the hardcoded oracle answers. Two details are load-bearing:

- **It renders results into the DOM**, not to `console.log()`, because page console output does not reach logcat in this build. Results are read with `adb shell screencap`.
- **Every async call is wrapped in a `Promise.race` timeout** (`T0()`). That is what turned "the page just sits there" into `getModel TIMEOUT` and made bug 2 visible at all. Keep it.

It also runs a **plain WebGPU baseline first** — `requestAdapter`, `requestDevice`, and a trivial `@compute @workgroup_size(1)` shader — so a failure can be blamed on the platform or on this module in one page load rather than one 40-minute reinstall.

`tools/serve_smoke_test.sh` serves the page from the **device's own 127.0.0.1**, a secure context. It pushes a pre-baked complete HTTP response (toybox `nc` cannot speak HTTP, so it just `cat`s a canned `resp.http`), runs `nc -4 -L -s 127.0.0.1 -p <port>` on the device, and holds the `adb shell` open because the listener dies with it. It calls `adb reverse --remove-all` first, since an `adb reverse` binds its listening socket **on the device** and collides with the port.

Result

From the 2026-08-23 session, against a build installed at 19:32:

```
requestAdapter -> adapter 79ms   vendor=qualcomm arch=adreno-8xx
requestDevice -> device 9ms
trivial WGSL compiled, msgs=0
navigator.digitclassifier: object
getModel OK 40ms
classify 1     -> "1" want "1" PASS 116ms
classify 7     -> "7" want "7" PASS 11ms
classify 0     -> "0" want "0" PASS 8ms
classify blank -> "5" want "5" PASS 6ms
=== 4/4 match the CPU oracle ===
```

The 116 ms first `classify()` versus 6-11 ms afterwards is pipeline creation and shader compilation happening once, on first dispatch.

9. Gotchas

Never measure on `out/Default`. It is a debug build, and a debug build was observed changing this GPU path's *classification answer*, not merely its timing. Anything you intend to measure or trust for correctness goes through `out/Release` (`is_debug=false`, `dcheck_always_on=false`, `symbol_level=1`). APK sizes differ accordingly: 715 MB debug, 390 MB release.

The Blink WTF/Oilpan boundary. Inside `third_party/blink/renderer/` use `blink::Vector`, `blink::String`, `Member<>/Persistent<>` — not STL containers or most `base/` equivalents. This module sits on the seam: `DigitClassifierWeights` stores a `Vector<float>` but hands out `base::span<const float>` because that is what Dawn wants.

-Wunsafe-buffer-usage rejects `memcpy` / `memcpy` / pointer arithmetic. Use `base::span` and `copy_from()`. Byte conversions of `float` additionally need the explicit `base::allow_nonunique_obj` tag.

Callback plumbing has three specific traps: it is `blink::BindOnce`, not `WTF::BindOnce`; `WrapCallbackInScriptScope()` injects the resolver as the first *unbound* parameter so every bound arg must precede it; and Dawn callbacks need `MakeWGPUOnceCallback(...)` plus `>UnboundCallback()` / `->AsUserdata()` and a matching `EnsureFlush()`.

Secure context. All three interfaces are `[SecureContext]`. `file://` is not secure, and `adb reverse` to a host server does not work when `adb` talks to a remote `adb` server over an SSH tunnel — the forward binds on the machine running the *adb server*. Serve from the device's own loopback.

An emulator will not validate this. `SwiftShader` or the host GPU would run the compute, so none of Dawn's Qualcomm workarounds are exercised.

Reaching the device is its own hazard. `adb` is not on `PATH`, the tunnel port moves, a dead tunnel hangs every `adb` command instead of failing it, and `bin/chrome_public_apk install` cannot work (devil caps `adb` at 300 s against a 715 MB APK). Relatedly, a backgrounded `autoninja ... | tail` reports the *pipeline's* exit code, so a failed build can look green.

10. Residual rough edges in the code

Not bugs that have bitten yet, but visible on a read:

- `DigitClassifierWeights::Load()` runs per `getModel()` call; nothing caches the model, so repeated calls re-copy 1.2 MB and build fresh GPU buffers.
- If the `CrossThreadHandle` unwraps in `EnsureDawnControlClient()` fail, the lambda returns without draining `dawn_control_client_ready_callbacks_`, leaving those promises pending — the exact failure signature of bug 2.
- `GpuDataManager::Are3DAPISBlockedForUrl()` is never consulted (§3.4).
- `TODO(crbug.com/None)` markers, no `OWNERS` file, no web tests — all three are required before this could land.

11. Where the existing docs are wrong

Verified against the code; in each case the code wins.

Claim	Where	Reality
"Can be used from renderer process."	base/android/apk_assets.h:18	False for a sandboxed Android renderer. <code>OpenApkAsset()</code> goes through JNI to the Java <code>AssetManager</code> (apk_assets.cc:29 , <code>Java_ApkAssets_open</code>), which an <code>isolatedProcess</code> cannot reach. This is precisely why the weights ship through blink_resources.grd .
"left uncompressed to keep navigator.digitclassifier's model load allocation-free"	blink_resources.grd:61-62	Not allocation-free. <code>UncompressResourceAsBinary()</code> calls <code>Platform::GetDataResourceString()</code> (a <code>std::string</code> copy out of the pak), copies that into a <code>Vector<char></code> (data_resource_helper.cc:24-29), and <code>Load()</code> copies again into a <code>Vector<float></code> — three ~1.2 MB allocations. Uncompressed buys a skipped decompression pass, nothing more.
reference_infer.py is a "CPU NumPy forward pass"	DigitClassifier-File-Inventory.md	It is pure stdlib Python with hand-rolled loops (<code>import math, struct, sys</code>). tflite_dump.py 's own docstring states the checkout has no <code>numpy</code> .
<code>navigator_digit_classifier.*</code> is a " <code>Supplement<Navigator></code> "	DigitClassifier-Handoff.md	It is <code>Supplement<NavigatorBase></code> (navigator_digit_classifier.h:17).
"nothing has ever executed on a GPU"	DigitClassifier-Handoff.md headline	Superseded — 4/4 against the oracle on an Adreno 830. The handoff banner says so; the line is still in the body.
Target device is a <i>OnePlus 13R</i> / <i>Snapdragon 8 Gen 3</i> / <i>Adreno 750</i>	DigitClassifier-Handoff.md	The device used was an OnePlus 13 (CPH2653), SM8750, Adreno 830 , Android 16. Immaterial in practice: Dawn's Qualcomm predicates <code>IsAndroidQualcomm()</code> and <code>MaybeQualcommProprietary()</code> key on <code>gpu_info::IsQualcommPCI(vendorId)</code> — vendor, not model .
Resume with <code>out/Default/bin/chrome_public_apk install</code>	DigitClassifier-Handoff.md	Cannot succeed over the current adb tunnel (devil's 300 s cap).
WGSL-Compilation-Path.md lives under <code>docs/digitclassifier/</code>	—	It contains zero mentions of this feature. It is general Dawn/Tint background, and it has an internal inconsistency of its own (prose says "six toggles" where its table lists seven rows). Useful, but not a document about this API.
<code>builtin_scalarize.cc:69</code> is where scalarization bails	DigitClassifier-Handoff.md §8	Substantively correct — the guard is <code>if (!builtin->Result()->Type()->Is<core::type::Vector>())</code> inside <code>ShouldAttemptScalarize()</code> — but the framing is Vulkan-only, and the toggle is also consumed by the D3D11, D3D12 and Metal backends.

Where the session log and the handoff disagree, the **session log wins**; it carries its own "Corrections to the handoff doc" section covering the last three handoff rows above.